In [1]:
```
# dictionary - used to store items as key:value pairs
# ordered (since Python 3.7), mutable, no duplicate keys
# dictionaries are indexed by keys, which can be any immutable type

student = {'roll': 5, "name": "Animesh", "age":22}
print(f"type= {type(student)}, student: {student}")
```

```
type= <class 'dict'>, student: {'roll': 5, 'name': 'Animesh', 'age': 22}
```

In [2]:
```
# no duplicate keys

student = {"roll": 5, "name": "Animesh", "age":22, "roll": 10}
print(f"student: {student}")
```

```
student: {'roll': 10, 'name': 'Animesh', 'age': 22}
```

In [3]:
```
# use of dict() to create dictionary

student = dict()
print(student)
```

```
{}
```

In [4]:
```
# dict() builds dictionaries directly from sequences of key-value pairs

student = dict([('roll', 5), ('name', "Animesh"), ('age', 22)])
print(student)
```

```
{'roll': 5, 'name': 'Animesh', 'age': 22}
```

In [5]:
```
# When the keys are simple strings, it is easier
# to specify pairs using keyword arguments

student = dict(roll=5, name = "Animesh", age = 22)
print(student)
```

```
{'roll': 5, 'name': 'Animesh', 'age': 22}
```

In [6]:
```
# len() function - how many items a dictionary has

len(student)
```

Out[6]:
```
3
```

In [7]:
```
# To check whether a single key is in the dictionary: in, not in

student = {'roll': 5, "name": "Animesh", "age":22}
print("roll" in student)
print("city" in student)
print("roll" not in student)
print("city" not in student)
```

```
True
False
False
True
```

In [8]:
```
# access values of an item with specified key
# key within square brackets

student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student['name'])
```

```
Animesh
```

In [9]:
```
# if key is not present, a KeyError occurs

student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student["email"])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[9], line 4
      1 # if key is not present, a KeyError occurs
      3 student = {'roll': 5, 'name': "Animesh", 'age': 22}
----> 4 print(student["email"])

KeyError: 'email'
```

In [10]:
```
help(dict.get)
```

```
Help on method_descriptor:

get(self, key, default=None, /)
    Return the value for key if key is in the dictionary, else default.
```

In [11]:
```
# access values of an item with specified key
# get() method

student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student.get("roll"))
```

```
5
```

In [12]:
```
student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student.get("city"))
```

```
None
```

In [13]:
```
# a value to return if the specified key does not exist

student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student.get("city","Kolkata"))
print(student)
```

```
Kolkata
{'roll': 5, 'name': 'Animesh', 'age': 22}
```

In [14]:
```
# traversing a dictionary
# value are the keys of the dictionary

student = {'roll': 5, 'name': "Animesh", 'age': 22}
for x in student:
    print(x)
```

```
roll
name
age
```

In [15]:
```
for x in student:
    print(x, student[x])
```

```
roll 5
name Animesh
age 22
```

In [17]:
```
help(dict.items)
```

```
Help on method_descriptor:

items(...)
    D.items() -> a set-like object providing a view on D's items
```

In [16]:
```
# items() method returns the key:value pair as list of tuples

student = {'roll': 5, 'name': "Ananya", 'age': 21}
print(student.items())
```

```
dict_items([('roll', 5), ('name', 'Ananya'), ('age', 21)])
```

In [17]:
```
for x in student.items():
    print(x)
```

```
('roll', 5)
('name', 'Ananya')
('age', 21)
```

In [18]:
```
for x, y in student.items():
    print(x,y)
```

```
roll 5
name Ananya
age 21
```

In [19]:
```
help(dict.keys)
```

```
Help on method_descriptor:

keys(...)
    D.keys() -> a set-like object providing a view on D's keys
```

In [20]:
```
# keys() method returns a view object
# the view object contains the keys of the dictionary, as a list.
# the view object will reflect any changes done to the dictionary

student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student.keys())
```

```
dict_keys(['roll', 'name', 'age'])
```

In [21]:
```
for x in student.keys():
    print(x)
```

```
roll
name
age
```

In [22]:
```
student = {"roll": 5, "name": "Animesh", "age":22}

x = student.keys()

student["email"] = "animesh.patra@heritageit.edu.in"

print(x)
```

```
dict_keys(['roll', 'name', 'age', 'email'])
```

In [23]:
```
help(dict.values)
```

```
Help on method_descriptor:

values(...)
    D.values() -> an object providing a view on D's values
```

In [24]:
```
# values() method returns a view object
# the view object contains the values of the dictionary, as a list
# the view object will reflect any changes done to the dictionary

student = {'roll': 5, 'name': "Animesh", 'age': 22}
print(student.values())
```

```
dict_values([5, 'Animesh', 22])
```

In [25]:
```
for x in student.values():
    print(x)
```

```
5
Animesh
22
```

In [26]:
```
student = {"roll": 5, "name": "Animesh", "age":22}
x = student.values()
student["email"] = "animesh.patra@heritageit.edu.in"
print(x)
```

```
dict_values([5, 'Animesh', 22, 'animesh.patra@heritageit.edu.in'])
```

In [27]:
```
# The objects returned by dict.keys(), dict.values() and dict.items()
# are view objects. They provide a dynamic view on the dictionary's entries,
# which means that when the dictionary changes, the view reflects these changes.
```

In [28]:
```
help(dict.pop)
help(dict.popitem)
help(dict.clear)
```

```
Help on method_descriptor:

pop(...)
    D.pop(k[,d]) -> v, remove specified key and return the corresponding value.

    If the key is not found, return the default if given; otherwise,
    raise a KeyError.

Help on method_descriptor:

popitem(self, /)
    Remove and return a (key, value) pair as a 2-tuple.

    Pairs are returned in LIFO (last-in, first-out) order.
    Raises KeyError if the dict is empty.

Help on method_descriptor:

clear(...)
    D.clear() -> None.  Remove all items from D.
```

In [29]:
```
# remove items from a dictionary
# pop(), popitem(), clear(), del

# pop() method removes the specified item from the dictionary

student = {"roll": 5, "name": "Animesh", "age":22}
```

```
x = student.pop("name")
print(f"Removed item: {x}")

print(f"Dictionary after removal: {student}")
```

```
Removed item: Animesh
Dictionary after removal: {'roll': 5, 'age': 22}
```

In [30]:
```
# popitem() method removes the item that was last inserted into the dictionary
# removed item is the return value of the popitem() method, as a tuple

student = {"roll": 5, "name": "Animesh", "age":22}
x = student.popitem()
print(f"Removed item: {x}")
print(f"Dictionary after removal: {student}")
```

```
Removed item: ('age', 22)
Dictionary after removal: {'roll': 5, 'name': 'Animesh'}
```

In [31]:
```
# del keyword removes the item with the specified key name
student = {"roll": 5, "name": "Animesh", "age":22}
del student["roll"]
print(f"Dictionary after removal: {student}")
```

```
Dictionary after removal: {'name': 'Animesh', 'age': 22}
```

In [32]:
```
# del keyword can also delete the dictionary completely

student = {"roll": 5, "name": "Animesh", "age":22}
del student
print(student)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[32], line 5
      3 student = {"roll": 5, "name": "Animesh", "age":22}
      4 del student
----> 5 print(student)

NameError: name 'student' is not defined
```

In [33]:
```
# clear() method empties the dictionary

student = {"roll": 5, "name": "Animesh", "age":22}
student.clear()
print(f"Dictionary after clear(): {student}")
```

```
Dictionary after clear(): {}
```

In [34]:
```
print(dir(dict))
```

```
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__',
'__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__get
item__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ior__', '__iter
__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__reduce__',
'__reduce_ex__', '__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem_
_', '__sizeof__', '__str__', '__subclasshook__', 'clear', 'copy', 'fromkeys', 'ge
t', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
```

In [35]:
```
help(dict.copy)
```

```
Help on method_descriptor:

copy(...)
    D.copy() -> a shallow copy of D
```

In [36]:
```
help(dict.fromkeys)
```

```
Help on built-in function fromkeys:

fromkeys(iterable, value=None, /) method of builtins.type instance
    Create a new dictionary with keys from iterable and values set to value.
```

In [37]:
```
x = dict.fromkeys([1,2,3])
y = dict.fromkeys((11,22,33), "mca")
print(f"x: {x}")
print(f"y: {y}")
```

```
x: {1: None, 2: None, 3: None}
y: {11: 'mca', 22: 'mca', 33: 'mca'}
```

In [38]:
```
help(dict.setdefault)
```

```
Help on method_descriptor:

setdefault(self, key, default=None, /)
    Insert key with a value of default if key is not in the dictionary.

    Return the value for key if key is in the dictionary, else default.
```

In [39]:
```
help(dict.update)
```

```
Help on method_descriptor:

update(...)
    D.update([E, ]**F) -> None.  Update D from dict/iterable E and F.
    If E is present and has a .keys() method, then does:  for k in E: D[k] = E[k]
    If E is present and lacks a .keys() method, then does:  for k, v in E: D[k] =
v
    In either case, this is followed by: for k in F:  D[k] = F[k]
```

In [40]:
```
# frequency of characters in a string
# Input: "students"
# Output: {'s': 2, 't': 2, 'u': 1, 'd': 1, 'e': 1, 'n': 1}
# Roll: 11

myStr = "students"
d = {}
for character in myStr:
    if character not in d:
        d[character] = 1
    else:
        d[character] = d[character] + 1
print(d)
```

```
{'s': 2, 't': 2, 'u': 1, 'd': 1, 'e': 1, 'n': 1}
```

In [42]:
```
#Roll: 13

myStr = "committee"
d = {}
for alphabet in myStr:
```

```
    d[alphabet] = d.get(alphabet,0) + 1
print(d)
```

```
{'c': 1, 'o': 1, 'm': 2, 'i': 1, 't': 2, 'e': 2}
```